

Proyecto II: Documentación Flujex

IC-4301 Bases de Datos I



Estudiante:

Josué Monterrey Hernández

Carné:

2025071773

Profesor:

Eddy Ramírez Jiménez

20 de mayo de 2026

Índice

1. Introducción	1
2. Diagrama Entidad-Relación	2
3. Diccionario de Datos	3
3.1. Tabla: <code>user</code>	3
3.2. Tabla: <code>currency</code>	3
3.3. Tabla: <code>account</code>	4
3.4. Tabla: <code>category</code>	4
3.5. Tabla: <code>budget</code>	5
3.6. Tabla: <code>exchange_rate</code>	5
3.7. Tabla: <code>transaction</code>	5
3.8. Tabla: <code>receipt</code>	6
3.9. Tabla: <code>external_source</code>	6
3.10. Tabla: <code>subscription</code>	7
4. Normalización	8
4.1. Primera Forma Normal (1FN)	8
4.2. Segunda Forma Normal (2FN)	8
4.3. Tercera Forma Normal (3FN)	9
4.4. Forma Normal de Boyce-Codd (FNBC)	9
4.5. Cuarta Forma Normal (4FN)	10
4.6. Quinta Forma Normal (5FN) - Forma de Proyección-Unión	10
4.7. Sexta Forma Normal (6FN)	11
4.8. Tabla Resumen de Normalización	12
5. Stored Procedures	13
5.1. SP-01: crear transacción	13
5.2. SP-02: adjuntar recibo	14
5.3. SP-03: procesar suscripciones pendientes	14
5.4. SP-04: desactivar usuario	15
5.5. SP-05: top de gastos	15
5.6. SP-06: resumen mensual	16
6. Triggers	17
6.1. TGR-01: actualizar saldos	17
6.2. TGR-02: actualizar presupuesto	17
6.3. TGR-03: registro de desactivación	18
7. Documentación Adicional	19
7.1. Gestión de Roles y Permisos	19
7.2. Especificaciones del Motor de Base de Datos	19
7.3. Plan de Respaldo (Backup)	20
8. Conclusiones	21
9. Referencias	22

1. Introducción

Flujex es una aplicación web de manejo de finanzas en la cuál el usuario puede:

- Abrir diversas cuentas de dinero.
- Realizar transacciones, incluso entre cuentas de distintas monedas.
- Registrar gastos e ingresos.
- Clasificar movimientos de dinero en categorías personalizadas.
- Establecer presupuestos.
- Registrar suscripciones y otros pagos automáticos periódicos.
- Observar un dashboard con un resumen y estadísticas de las finanzas de sus cuentas.

Esta aplicación fue desarrollada utilizando React+Vite para el *front-end* y Python3 para el *back-end*. Además, se utilizó MySQL como sistema de gestión de bases de datos.

El presente documento se centra en explicar las características, estructura y elementos de la base de datos relacional de la aplicación [1], así como su normalización hasta la sexta forma normal (6FN) y el diccionario de datos. Asimismo, se explican temas de seguridad, usuarios y roles, el plan de respaldo de datos, stored procedures, triggers y otros aspectos.

2. Diagrama Entidad-Relación

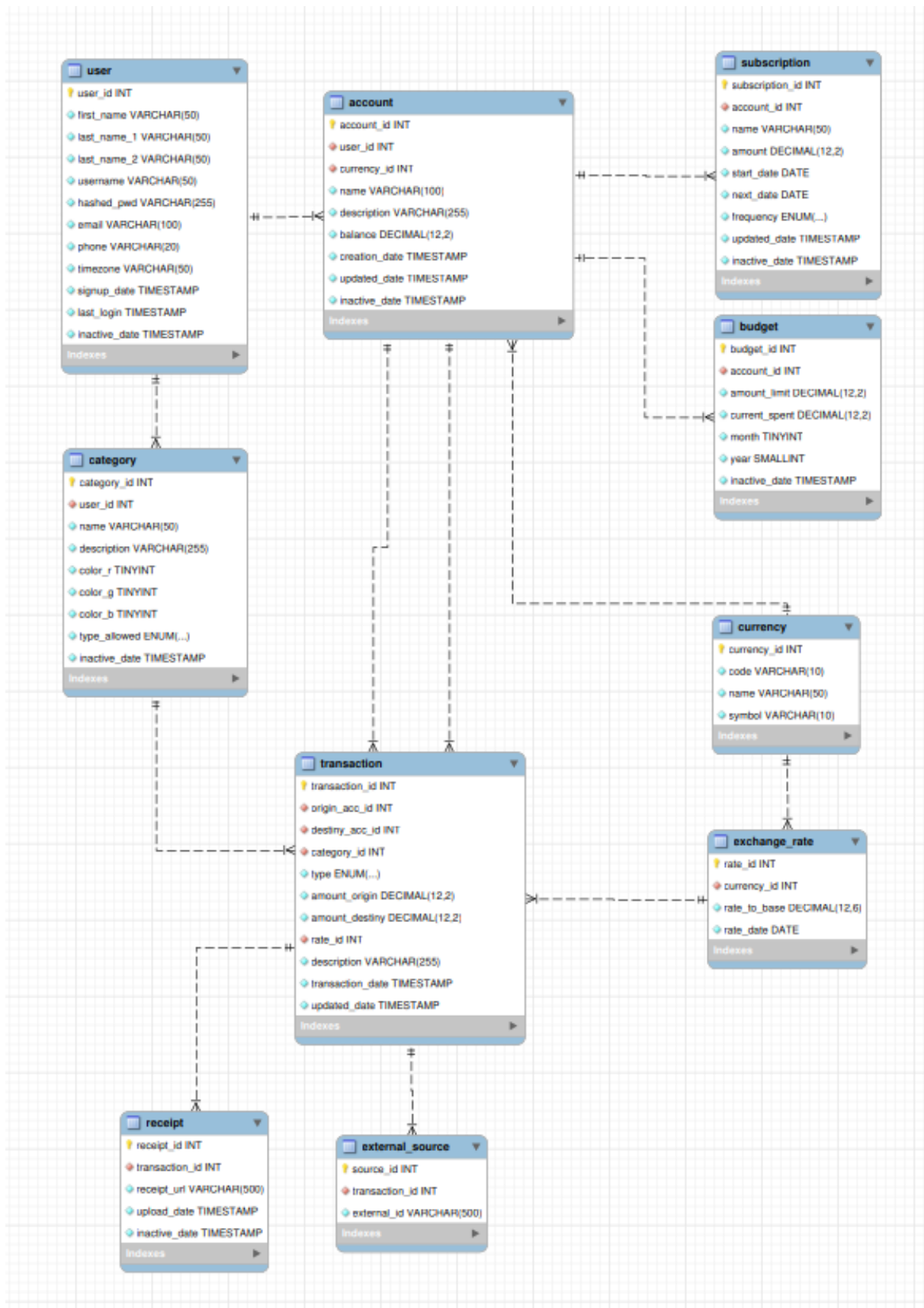


Figura 1: Diagrama Entidad-Relación de la base de datos Flujex

3. Diccionario de Datos

3.1. Tabla: user

Campo	Nombre Lógico	Tipo	Long	Nulo	Llave	Default	Descripción
user_id	ID Usuario	INT	-	NO	PK	AUTO.INCREMENT	Identificador único.
first_name	Primer Nombre	VARCHAR	50	NO	-	-	Nombre del usuario.
last_name_1	Primer Apellido	VARCHAR	50	NO	-	-	Primer apellido.
last_name_2	Segundo Apellido	VARCHAR	50	NO	-	-	Segundo apellido.
username	Nombre de Usuario	VARCHAR	50	NO	-	-	Nickname único.
hashed_pwd	Contraseña Hash	VARCHAR	255	NO	-	-	Contraseña encriptada.
email	Correo Electrónico	VARCHAR	100	NO	-	-	Correo electrónico.
phone	Teléfono	VARCHAR	20	NO	-	-	Número telefónico.
timezone	Zona Horaria	VARCHAR	50	NO	-	'UTC'	Configuración horaria.
signup_date	Fecha Registro	TIMESTAMP	-	NO	-	CURRENT_TIMESTAMP	Fecha de creación.
last_login	Último Acceso	TIMESTAMP	-	NO	-	'2038-01-01 00:00:00'	Última sesión.
inactive_date	Fecha Inactivo	TIMESTAMP	-	NO	-	'2038-01-01 00:00:00'	Fecha de borrado lógico.

Relaciones:

Ninguna (tabla raíz).

Restricciones adicionales:

- UNIQUE KEY (email, inactive_date)
- INDEX idx_email ON (email)

3.2. Tabla: currency

Campo	Nombre Lógico	Tipo	Long	Nulo	Llave	Default	Descripción
currency_id	ID Moneda	INT	-	NO	PK	AUTO_INC	Identificador único.
code	Código Divisa	VARCHAR	10	NO	UNIQUE	-	Código ISO.
name	Nombre Divisa	VARCHAR	50	NO	-	-	Nombre de la moneda.
symbol	Símbolo	VARCHAR	10	NO	-	-	Símbolo monetario.

Relaciones:

Ninguna (tabla raíz).

Restricciones adicionales:

- UNIQUE KEY (code)
- INDEX idx_code ON (code)

3.3. Tabla: account

Campo	Nombre Lógico	Tipo	Long	Nulo	Llave	Default	Descripción
account_id	ID Cuenta	INT	-	NO	PK	AUTO_INC	Identificador único.
user_id	ID Usuario	INT	-	NO	FK	-	Propietario de la cuenta.
currency_id	ID Moneda	INT	-	NO	FK	-	Divisa asociada.
name	Nombre Cuenta	VARCHAR	100	NO	-	-	Nombre personalizado.
description	Descripción	VARCHAR	255	NO	-	-	Detalles adicionales.
balance	Saldo Actual	DECIMAL	12,2	NO	-	0.00	Fondos disponibles.
creation_date	Fecha Creación	TIMESTAMP	-	NO	-	CURRENT_TIMESTAMP	Registro en el sistema.
updated_date	Fecha Modificación	TIMESTAMP	-	NO	-	CURRENT_TIMESTAMP	Auto-actualizable.
inactive_date	Fecha Inactivo	TIMESTAMP	-	NO	-	'2038-01-01 00:00:00'	Borrado lógico.

Relaciones:

Campo origen	Tabla ref.	Campo ref.	Cardinalidad	ON DELETE	ON UPDATE
user_id	user	user_id	Muchos a Uno	CASCADE	RESTRICT
currency_id	currency	currency_id	Muchos a Uno	RESTRICT	RESTRICT

Restricciones adicionales:

- INDEX idx_user_acc ON (user_id)

3.4. Tabla: category

Campo	Nombre Lógico	Tipo	Long	Nulo	Llave	Default	Descripción
category_id	ID Categoría	INT	-	NO	PK	AUTO_INC	Identificador único.
user_id	ID Usuario	INT	-	NO	FK	-	Creador de la categoría.
name	Nombre	VARCHAR	50	NO	-	-	Nombre de categoría.
description	Descripción	VARCHAR	255	NO	-	-	Notas de la categoría.
color_r	Canal Rojo	TINYINT	-	NO	-	0	Color RGB - Rojo.
color_g	Canal Verde	TINYINT	-	NO	-	0	Color RGB - Verde.
color_b	Canal Azul	TINYINT	-	NO	-	0	Color RGB - Azul.
type_allowed	Tipos Admitidos	ENUM	-	NO	-	'Both'	Deposit, Expense, Both.
inactive_date	Fecha Inactivo	TIMESTAMP	-	NO	-	'2038-01-01 00:00:00'	Borrado lógico.

Relaciones:

Campo origen	Tabla ref.	Campo ref.	Cardinalidad	ON DELETE	ON UPDATE
user_id	user	user_id	Muchos a Uno	CASCADE	RESTRICT

Restricciones adicionales:

- INDEX idx_usr_category ON (user_id)

3.5. Tabla: budget

Campo	Nombre Lógico	Tipo	Long	Nulo	Llave	Default	Descripción
budget_id	ID Presupuesto	INT	-	NO	PK	AUTO_INC	Identificador único.
account_id	ID Cuenta	INT	-	NO	FK	-	Cuenta bajo presupuesto.
amount_limit	Límite Dinero	DECIMAL	12,2	NO	-	-	Presupuesto asignado.
current_spent	Consumido	DECIMAL	12,2	NO	-	0.00	Gasto acumulado del mes.
month	Mes del Año	TINYINT	-	NO	-	-	Rango del 1 al 12.
year	Año numérico	SMALLINT	-	NO	-	-	Año asignado.
inactive_date	Fecha Inactivo	TIMESTAMP	-	NO	-	'2038-01-01 00:00:00'	Borrado lógico.

Relaciones:

Campo origen	Tabla ref.	Campo ref.	Cardinalidad	ON DELETE	ON UPDATE
account_id	account	account_id	Muchos a Uno	CASCADE	RESTRICT

Restricciones adicionales:

- CHECK (month BETWEEN 1 AND 12)
- UNIQUE KEY budget_period (account_id, month, year, inactive_date)

3.6. Tabla: exchange_rate

Campo	Nombre Lógico	Tipo	Long	Nulo	Llave	Default	Descripción
rate_id	ID Tasa Cambio	INT	-	NO	PK	AUTO_INC	Identificador único.
currency_id	ID Moneda	INT	-	NO	FK	-	Divisa evaluada.
rate_to_base	Multiplicador Base	DECIMAL	12,6	NO	-	-	Tasa con respecto a base.
rate_date	Fecha Tasa	DATE	-	NO	-	CURRENT_DATE	Día del registro de tasa.

Relaciones:

Campo origen	Tabla ref.	Campo ref.	Cardinalidad	ON DELETE	ON UPDATE
currency_id	currency	currency_id	Muchos a Uno	RESTRICT	RESTRICT

Restricciones adicionales:

- UNIQUE KEY (currency_id, rate_date)

3.7. Tabla: transaction

Campo	Nombre Lógico	Tipo	Long	Nulo	Llave	Default	Descripción
transaction_id	ID Transacción	INT	-	NO	PK	AUTO_INC	Identificador único.
origin_acc_id	ID Cuenta Origen	INT	-	NO	FK	-	Cuenta de donde sale dinero.
destiny_acc_id	ID Cuenta Destino	INT	-	NO	FK	-	Cuenta que recibe fondos.
category_id	ID Categoría	INT	-	NO	FK	-	Categoría de la transac.
type	Tipo Movimiento	ENUM	-	NO	-	'Transfer'	Expense, Deposit, Transfer.
amount_origin	Monto Origen	DECIMAL	12,2	NO	-	-	Dinero debitado de origen.
amount_destiny	Monto Destino	DECIMAL	12,2	NO	-	-	Dinero acreditado a destino.
rate_id	ID Tasa Cambio	INT	-	NO	FK	-	Tasa de cambio aplicada.
description	Detalle	VARCHAR	255	NO	-	-	Descripción.
transaction_date	Fecha Ejecución	TIMESTAMP	-	NO	-	CURRENT_TIMESTAMP	Momento de la operación.
updated_date	Fecha Modificado	TIMESTAMP	-	NO	-	CURRENT_TIMESTAMP	Momento de actualización.

Relaciones:

Campo origen	Tabla ref.	Campo ref.	Cardinalidad	ON DELETE	ON UPDATE
origin_acc_id	account	account_id	Muchos a Uno	RESTRICT	RESTRICT
destiny_acc_id	account	account_id	Muchos a Uno	RESTRICT	RESTRICT
category_id	category	category_id	Muchos a Uno	RESTRICT	RESTRICT
rate_id	exchange_rate	rate_id	Muchos a Uno	RESTRICT	RESTRICT

Restricciones adicionales:

- INDEX idx_date_search ON (transaction_date)
- INDEX idx_acc_history ON (origin_acc_id, transaction_date)

3.8. Tabla: receipt

Campo	Nombre Lógico	Tipo	Long	Nulo	Llave	Default	Descripción
receipt_id	ID Recibo	INT	-	NO	PK	AUTO_INC	Identificador único.
transaction_id	ID Transacción	INT	-	NO	FK	-	Transacción asociada.
receipt_url	URL del Archivo	VARCHAR	500	NO	-	-	Path de la imagen.
upload_date	Fecha Subida	TIMESTAMP	-	NO	-	-	Momento de la carga.
inactive_date	Fecha Inactivo	TIMESTAMP	-	NO	-	'2038-01-01 00:00:00'	Borrado lógico.

Relaciones:

Campo origen	Tabla ref.	Campo ref.	Cardinalidad	ON DELETE	ON UPDATE
transaction_id	transaction	transaction_id	Uno a Uno	CASCADE	RESTRICT

3.9. Tabla: external_source

Campo	Nombre Lógico	Tipo	Long	Nulo	Llave	Default	Descripción
source_id	ID Origen Ext	INT	-	NO	PK	AUTO_INC	Identificador único.
transaction_id	ID Transacción	INT	-	NO	FK	-	Transacción asociada.
external_id	ID Externo	VARCHAR	500	NO	-	-	ID del banco / API externa.

Relaciones:

Campo origen	Tabla ref.	Campo ref.	Cardinalidad	ON DELETE	ON UPDATE
transaction_id	transaction	transaction_id	Uno a Uno	CASCADE	RESTRICT

3.10. Tabla: subscription

Campo	Nombre Lógico	Tipo	Long	Nulo	Llave	Default	Descripción
subscription_id	ID Suscripción	INT	-	NO	PK	AUTO_INC	Identificador único.
account_id	ID Cuenta	INT	-	NO	FK	-	Cuenta que paga débito.
name	Nombre Servicio	VARCHAR	50	NO	-	-	Ej. Netflix, Spotify.
amount	Monto Periódico	DECIMAL	12,2	NO	-	-	Tarifa fija del servicio.
start_date	Fecha Inicial	DATE	-	NO	-	CURRENT_DATE	Comienzo del cobro.
next_date	Próxima Fecha	DATE	-	NO	-	CURRENT_DATE	Siguiente renovación.
frequency	Frecuencia Pago	ENUM	-	NO	-	-	Daily, Weekly, Monthly...
updated_date	Fecha Cambios	TIMESTAMP	-	NO	-	CURRENT_TIMESTAMP	Fecha de actualización.
inactive_date	Fecha Inactivo	TIMESTAMP	-	NO	-	'2038-01-01 00:00:00'	Cancelación del servicio.

Relaciones:

Campo origen	Tabla ref.	Campo ref.	Cardinalidad	ON DELETE	ON UPDATE
account_id	account	account_id	Muchos a Uno	RESTRICT	RESTRICT

Restricciones adicionales:

- INDEX idx_subs_next ON (next_date)

4. Normalización

En esta sección se demuestra formalmente que el diseño de la base de datos cumple con el proceso de normalización desde la primera forma normal (1FN) hasta la sexta forma normal (6FN) [2].

4.1. Primera Forma Normal (1FN)

Definición: Una relación está en 1FN si:

- Todos los atributos contienen valores atómicos.
- No existen grupos repetitivos de columnas.
- Existe una llave primaria claramente definida.

Demostración sobre el sistema:

- **Atomicidad:** Todos los campos de texto, numéricos y de fecha almacenan un único dato por registro. Por ejemplo, en la tabla `user`, los campos `first_name`, `last_name_1` y `last_name_2` están separados en columnas independientes en lugar de usar un único campo compuesto de nombre completo. De igual forma, el campo `email` almacena una sola dirección por cuenta.
- **Ausencia de grupos repetitivos:** Ninguna tabla posee columnas indexadas secuencialmente para evadir relaciones (como `phone1`, `phone2` en `user`, o `category_id1`, `category_id2` en `transaction`). Cuando se requiere multiplicidad, se emplean tablas intermedias o tablas hijas.
- **Llave Primaria:** Cada una de las 10 entidades posee una clave primaria con la propiedad `AUTO_INCREMENT`.

✓ *Todas las tablas del sistema cumplen con la 1FN.*

4.2. Segunda Forma Normal (2FN)

Definición: Una relación está en 2FN si:

- Está en 1FN.
- Todos los atributos no clave dependen completamente de la llave primaria (sin dependencias parciales). Solo es relevante cuando la PK es compuesta.

Demostración sobre el sistema:

- **Llaves primarias simples:** Todas las entidades cuentan con una llave primaria simple con un valor autoincremental, por ende no existen dependencias parciales.

✓ *Todas las tablas del sistema cumplen con la 2FN.*

4.3. Tercera Forma Normal (3FN)

Definición: Una relación está en 3FN si:

- Está en 2FN.
- No existen dependencias transitivas: ningún atributo no clave depende de otro atributo no clave.

Demostración sobre el sistema:

- **Aislamiento de entidades independientes:** Se crearon tablas hijas como `currency` para extraer los datos del nombre, símbolo y código de las divisas y evitar que dependieran de `currency_id` y `account_id` al mismo tiempo. De este modo, la información se encuentra correctamente separada en entidades independientes, evitando redundancias.

✓ *Todas las tablas del sistema cumplen con la 3FN.*

4.4. Forma Normal de Boyce-Codd (FNBC)

Definición: Una relación está en FNBC si:

- Está en 3FN.
- Para toda dependencia funcional $X \rightarrow Y$, X es una superllave.

Demostración sobre el sistema: Al analizar las tablas con múltiples llaves candidatas o restricciones únicas se observa:

- **Tabla user:** Las dependencias funcionales son:

- `user_id` $\rightarrow$ todo
- `(email, inactive_date)` $\rightarrow$ todo

Todos los determinantes son superclaves.

- **Tabla currency:** Las dependencias funcionales son:

- `currency_id` $\rightarrow$ todo
- `code` $\rightarrow$ todo

Todos los determinantes son superclaves.

- **Tabla budget:** Las dependencias funcionales son:

- `budget_id` $\rightarrow$ todo
- `(account_id, month, year, inactive_date)` $\rightarrow$ todo

Todos los determinantes son superclaves.

- **Tabla exchange_rate:** Las dependencias funcionales son:

- $\text{rate_id} \rightarrow \text{todo}$
- $(\text{currency_id}, \text{rate_date}) \rightarrow \text{todo}$

Todos los determinantes son superclaves.

- **Tabla receipt:** Las dependencias funcionales son:

- $\text{receipt_id} \rightarrow \text{todo}$
- $\text{transaction_id} \rightarrow \text{todo}$

Todos los determinantes son superclaves.

- **Tabla external_source:** Las dependencias funcionales son:

- $\text{source_id} \rightarrow \text{todo}$
- $\text{transaction_id} \rightarrow \text{todo}$

Todos los determinantes son superclaves.

✓ *Todas las tablas del sistema cumplen con la FNBC.*

4.5. Cuarta Forma Normal (4FN)

Definición: Una relación está en 4FN si:

- Está en FNBC.
- No existen dependencias multivaluadas (MVD) no triviales.

Demostración sobre el sistema:

- Ya que las dependencias entre entidades fueron separadas adecuadamente en tablas independientes no existen atributos multivaluados.

✓ *Todas las tablas del sistema cumplen con la 4FN.*

4.6. Quinta Forma Normal (5FN) - Forma de Proyección-Unión

Definición: Una relación está en 5FN (también llamada *PJ/NF* — *Project-Join Normal Form*) si:

- Está en 4FN.
- No puede descomponerse en proyecciones más pequeñas sin pérdida de información, a menos que las proyecciones estén basadas en superllaves (dependencias de unión triviales).

Demostración sobre el sistema:

- Las relaciones del sistema representan datos atómicos, por lo que no existen dependencias de unión no triviales. No hay relaciones complejas que puedan descomponerse adicionalmente.

✓ *Las relaciones del sistema son inherentemente acíclicas, satisfaciendo la 5FN.*

4.7. Sexta Forma Normal (6FN)

Definición: Una relación está en 6FN si:

- Está en 5FN.
- Cada tabla contiene únicamente la llave primaria y un solo atributo no clave (descomposición máxima).
- Se elimina toda dependencia de unión, incluyendo aquellas relacionadas con datos temporales o intervalos.

Demostración sobre el sistema:

- El sistema actual no maneja componentes temporales complejos ni intervalos de vigencia independientes.
- Los datos financieros, como balances, presupuestos y montos de transacciones, se almacenan como valores atómicos asociados a un único evento del sistema.
- Las fechas registradas (transaction_date, signup_date, updated_date, rate_date) representan atributos simples y no generan dependencias temporales independientes.
- No existen atributos históricos que requieran descomposición adicional en tablas separadas por vigencia o intervalos temporales.
- En un escenario futuro, si se implementaran historiales completos de balances, tipos de cambio o presupuestos con periodos de vigencia independientes, podrían requerirse descomposiciones adicionales siguiendo los principios de 6FN.

✓ *Para el alcance actual del proyecto, el diseño cumple la 6FN por simplicidad estructural y ausencia de componentes temporales complejos.*

4.8. Tabla Resumen de Normalización

A continuación, se muestra el estado de cumplimiento de normalización de cada entidad del esquema de la base de datos:

Tabla	1FN	2FN	3FN	FNBC	4FN	5FN	6FN	Observación
user	✓	✓	✓	✓	✓	✓	✓	Manejo principal de usuarios y autenticación.
currency	✓	✓	✓	✓	✓	✓	✓	Información básica de monedas del sistema.
account	✓	✓	✓	✓	✓	✓	✓	Registro de cuentas financieras por usuario.
category	✓	✓	✓	✓	✓	✓	✓	Clasificación de transacciones y presupuestos.
budget	✓	✓	✓	✓	✓	✓	✓	Control periódico de límites de gasto.
exchange_rate	✓	✓	✓	✓	✓	✓	✓	Registro diario de tipos de cambio.
transaction	✓	✓	✓	✓	✓	✓	✓	Registro principal de movimientos financieros.
receipt	✓	✓	✓	✓	✓	✓	✓	Almacenamiento de comprobantes asociados.
external_source	✓	✓	✓	✓	✓	✓	✓	Referencias externas de transacciones.
subscription	✓	✓	✓	✓	✓	✓	✓	Gestión de pagos y cargos recurrentes.

5. Stored Procedures

Los **stored procedures** permiten almacenar consultas bajo un nombre específico y ejecutarlas mediante una llamada (**CALL**).

Con la intención de simplificar las acciones de la aplicación se modelaron seis stored procedures siguiendo la sintaxis estándar de MySQL 8.0 [3]:

5.1. SP-01: crear transacción

Descripción: Automatiza el proceso de transferir dinero de una cuenta origen a una cuenta destino. Actualiza el saldo de ambas cuentas y registra el movimiento en la tabla **transaction**.

```
1 DELIMITER //
```

```
2 CREATE PROCEDURE sp_create_transaction (  
3     IN p_origin_acc_id INT,  
4     IN p_destiny_acc_id INT,  
5     IN p_category_id INT,  
6     IN p_type ENUM ('Expense', 'Deposit', 'Transfer'),  
7     IN p_amount_origin DECIMAL(12, 2),  
8     IN p_amount_destiny DECIMAL(12, 2)  
9 ) BEGIN  
10     UPDATE account  
11     SET balance = balance - p_amount_origin  
12     WHERE account_id = p_origin_acc_id;  
13  
14     UPDATE account  
15     SET balance = balance + p_amount_destiny  
16     WHERE account_id = p_destiny_acc_id;  
17  
18     INSERT INTO transaction (  
19         origin_acc_id,  
20         destiny_acc_id,  
21         category_id,  
22         type,  
23         amount_origin,  
24         amount_destiny  
25     )  
26     VALUES (  
27         p_origin_acc_id,  
28         p_destiny_acc_id,  
29         p_category_id,  
30         p_type,  
31         p_amount_origin,  
32         p_amount_destiny  
33     );  
34  
35     COMMIT;  
36  
37 END //
```

```
38 DELIMITER ;
```

5.2. SP-02: adjuntar recibo

Descripción: Permite rápidamente adjuntar una factura o recibo a una transacción específica.

```
1 DELIMITER //
```

```
2 CREATE PROCEDURE sp_add_receipt (  
3     IN p_transaction_id INT,  
4     IN p_receipt_url VARCHAR(500)  
5 ) BEGIN  
6     INSERT INTO  
7         receipt (transaction_id, receipt_url)  
8     VALUES  
9         (p_transaction_id, p_receipt_url);  
10  
11 END //
```

```
12 DELIMITER ;
```

5.3. SP-03: procesar suscripciones pendientes

Descripción: Rebaja el monto correcto para todas las cuentas con suscripciones pendientes de pagar y actualiza la siguiente fecha de cobro.

```
1 DELIMITER //
```

```
2 CREATE PROCEDURE sp_process_due_subscriptions ()  
3 BEGIN  
4     UPDATE account a  
5     JOIN subscription s  
6         ON a.account_id = s.account_id  
7     SET a.balance = a.balance - s.amount  
8     WHERE s.next_date <= CURRENT_DATE();  
9  
10    UPDATE subscription  
11    SET next_date = DATE_ADD(next_date, INTERVAL 1 MONTH)  
12    WHERE next_date <= CURRENT_DATE();  
13  
14 END //
```

```
15 DELIMITER ;
```


5.4. SP-04: desactivar usuario

Descripción: Realiza un borrado lógico a un usuario y sus cuentas actualizando el campo `inactive_date`.

```
1 DELIMITER //
```

```
2 CREATE PROCEDURE sp_deactivate_user (IN p_user_id INT)
```

```
3 BEGIN
```

```
4     UPDATE user
```

```
5     SET inactive_date = NOW()
```

```
6     WHERE user_id = p_user_id;
```

```
7
```

```
8     UPDATE account
```

```
9     SET inactive_date = NOW()
```

```
10    WHERE user_id = p_user_id;
```

```
11
```

```
12 END //
```

```
13 DELIMITER ;
```

5.5. SP-05: top de gastos

Descripción: Obtiene el top 5 de las categorías más utilizadas en gastos.

```
1 DELIMITER //
```

```
2 CREATE PROCEDURE sp_get_top_expense_categories (IN p_user_id INT)
```

```
3 BEGIN
```

```
4     SELECT
```

```
5         c.name,
```

```
6         SUM(t.amount_origin) AS total_spent
```

```
7     FROM transaction t
```

```
8     JOIN category c
```

```
9         ON t.category_id = c.category_id
```

```
10    JOIN account a
```

```
11        ON t.origin_acc_id = a.account_id
```

```
12    WHERE a.user_id = p_user_id
```

```
13        AND t.type = 'Expense'
```

```
14    GROUP BY c.name
```

```
15    ORDER BY total_spent DESC
```

```
16    LIMIT 5;
```

```
17
```

```
18 END //
```

```
19 DELIMITER ;
```

5.6. SP-06: resumen mensual

Descripción: Calcula el total de gastos e ingresos de una cuenta durante el mes actual.

```
1 DELIMITER //
```

```
2 CREATE PROCEDURE sp_get_monthly_summary (  
3     IN p_user_id INT,  
4     IN p_month INT,  
5     IN p_year INT  
6 )  
7 BEGIN  
8     SELECT  
9         SUM(  
10            CASE  
11                WHEN t.type = 'Deposit' THEN t.amount_destiny  
12            ELSE 0  
13            END  
14        ) AS total_income,  
15        SUM(  
16            CASE  
17                WHEN t.type = 'Expense' THEN t.amount_origin  
18            ELSE 0  
19            END  
20        ) AS total_expenses  
21 FROM transaction t  
22 JOIN account a  
23     ON t.origin_acc_id = a.account_id  
24 WHERE a.user_id = p_user_id  
25        AND MONTH (t.transaction_date) = p_month  
26        AND YEAR (t.transaction_date) = p_year;  
27  
28 END //
```

```
29 DELIMITER ;
```

6. Triggers

Los **triggers** son objetos de las bases de datos que ejecutan automáticamente un conjunto de consultas cuando ocurre un evento específico en una tabla concreta.

Con el fin de automatizar y agilizar ciertos procesos del sistema se diseñaron tres triggers siguiendo la sintaxis estándar de MySQL 8.0 [4]:

6.1. TGR-01: actualizar saldos

Descripción: En el momento en que se registra una nueva transacción este trigger actualiza automáticamente el saldo de la cuenta origen y destino.

```
1 CREATE TRIGGER trg_update_balance_after_transaction AFTER INSERT
2 ON transaction
3 FOR EACH ROW
4 BEGIN
5     UPDATE account
6     SET balance = balance - NEW.amount_origin
7     WHERE account_id = NEW.origin_acc_id;
8
9     UPDATE account
10    SET balance = balance + NEW.amount_destiny
11    WHERE account_id = NEW.destiny_acc_id;
12 END;
```

6.2. TGR-02: actualizar presupuesto

Descripción: En el momento en que se registra una nueva transacción este trigger actualiza el saldo restante de cualquier presupuesto asociado a la cuenta.

```
1 CREATE TRIGGER trg_update_budget_after_expense AFTER INSERT
2 ON transaction
3 FOR EACH ROW
4 BEGIN
5     IF NEW.type = 'Expense' THEN
6         UPDATE budget
7         SET current_spent = current_spent + NEW.amount_origin
8         WHERE
9             account_id = NEW.origin_acc_id
10            AND month = MONTH (CURRENT_DATE)
11            AND year = YEAR (CURRENT_DATE);
12     END IF;
13 END;
```

6.3. TGR-03: registro de desactivación

Descripción: Con la ayuda de una tabla extra `audit_log` se registran los cambios realizados a una cuenta al momento de su desactivación.

```
1 CREATE TRIGGER trg_audit_user_deactivation AFTER UPDATE
2 ON user FOR EACH ROW
3 BEGIN
4     IF OLD.inactive_date = '2038-01-01 00:00:00'
5     AND NEW.inactive_date <= NOW() THEN
6         INSERT INTO audit_log (action_type, table_name, record_id)
7         VALUES (
8             'DEACTIVATE',
9             'user',
10            NEW.user_id
11        );
12     END IF;
13 END;
```

7. Documentación Adicional

7.1. Gestión de Roles y Permisos

Para el acceso y manejo de operaciones dentro de la base de datos se crearon dos usuarios con sus respectivos roles y permisos: `admin` y `app`.

```
1 DROP ROLE IF EXISTS 'admin';
2 DROP ROLE IF EXISTS 'app';
3
4 CREATE ROLE 'admin';
5 CREATE ROLE 'app';
6
7 GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, DROP
8   ON * TO 'admin';
9 GRANT SELECT, INSERT, UPDATE
10  ON * TO 'app';
11
12 DROP USER IF EXISTS 'flujex_admin'@'localhost';
13 CREATE USER 'flujex_admin'@'localhost' IDENTIFIED BY 'ADMIN_PWD';
14 GRANT 'admin' TO 'flujex_admin'@'localhost';
15 ALTER USER 'flujex_admin'@'localhost' DEFAULT ROLE 'admin';
16
17 DROP USER IF EXISTS 'flujex_app'@'localhost';
18 CREATE USER 'flujex_app'@'localhost' IDENTIFIED BY 'APP_PWD';
19 GRANT 'app' TO 'flujex_app'@'localhost';
20 ALTER USER 'flujex_app'@'localhost' DEFAULT ROLE 'app';
21
22 FLUSH PRIVILEGES;
```

Como se puede apreciar, el usuario `flujex_admin` tiene permisos para seleccionar, insertar, actualizar y eliminar en todas las tablas de la base de datos. Además, puede crear, alterar o eliminar tablas. El usuario `flujex_app`, por otro lado, solamente es capaz de seleccionar, insertar y actualizar registros en las tablas de la base de datos.

El usuario `flujex_app` no requiere permisos para eliminar registros ya que todos los datos insertados son permanentes y su eliminación es únicamente lógica utilizando una fecha de desactivación.

7.2. Especificaciones del Motor de Base de Datos

Parámetro	Valor
Motor de BD	MySQL 8.0
Motor de almacenamiento	InnoDB
Charset	utf8mb4
Collation	utf8mb4_0900_ai_ci
Zona horaria	System
Puerto	3306
Nombre de la BD	flujex
Nivel de aislamiento	REPEATABLE-READ

7.3. Plan de Respaldo (Backup)

Aspecto	Detalle
Herramienta	mysqldump
Tipo de respaldo	Completo semanal + Incremental diario (binlog)
Frecuencia	Completo: Domingo 02:00 AM / Incremental: Diario 02:00 AM
Ubicación	Servidor local + Réplica en almacenamiento en la nube
Retención	30 días para completos / 7 días para incrementales
RTO (Tiempo recuperación)	Máximo 2 horas
RPO (Pérdida máxima)	Máximo 24 horas
Verificación	Restauración de prueba mensual en entorno de staging

8. Conclusiones

Se desarrolló una aplicación web responsive de manejo de finanzas que permite al usuario monitorear, registrar y categorizar los movimientos que realiza su dinero. Gracias a la posibilidad de separar su dinero en diversas cuentas contables y establecer presupuestos el usuario evita gastar más dinero del necesario para así alcanzar sus metas de ahorro. Además, la aplicación le permite tener un listado de los pagos periódicos (como suscripciones y matrículas) y cobra los montos establecidos en los intervalos de tiempo que el usuario indique.

Todos los datos que maneja la aplicación se encuentran almacenados en una base de datos relacional correctamente normalizada desde la 1FN hasta la 6FN, lo que evita redundancias de datos, conflictos al momento de insertar, actualizar o eliminar datos, y asegura la integridad de la información. Además, por razones de seguridad, se crearon dos usuarios para el manejo de la base de datos: un administrador que tiene todos los permisos para manipular los datos y crear, alterar y eliminar tablas, y un usuario de aplicación que solamente tiene permisos para seleccionar, insertar y actualizar datos.

Se plantearon seis stored procedures que agilizan algunos procesos recurrentes que realiza la aplicación como crear transacciones, procesar suscripciones pendientes u obtener un top de gastos por categorías). Además, se crearon tres triggers que ejecutan procesos automáticamente para asegurar la actualización o auditoría de datos inmediatamente después de una transacción o eliminación.

La aplicación tiene muchas oportunidades de mejora y expansión de funciones. Por ejemplo, se planea implementar un sistema de registro automático de transacciones utilizando un agente de inteligencia artificial que analice los correos electrónicos del usuario. Junto a esto se desea mejorar el sistema para permitir a los usuarios adjuntar un recibo físico o digital a las transacciones realizadas para tener pruebas de que el movimiento verdaderamente ocurrió.

9. Referencias

- [1] E. F. Codd, “A relational model of data for large shared data banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, jun 1970.
- [2] IBM. (2026) What is database normalization? [Online]. Available: <https://www.ibm.com/think/topics/database-normalization>
- [3] W3Schools. Mysql stored procedures. [Online]. Available: https://www.w3schools.com/mysql/mysql_stored_procedures.asp
- [4] Oracle. Mysql 8.4 reference manual: Trigger syntax and examples. [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/trigger-syntax.html>